

FoldPulse

Every Fold. Every Beat.

PHASE 1 · PRODUCT RESEARCH & FEASIBILITY

Feasibility Report & Recommendation

Version	1.0
Date	9 November 2026
Prepared for	Founder & Product Owner
Prepared by	FoldPulse product team (12 agent roles)
Status	Submitted for Founder approval
Recommendation	**CONDITIONAL GO**

Confidential — prepared for the FoldPulse Founder. Icon shown is a temporary concept.

Contents

1	Executive summary & recommendation	2
2	Team, environment & method	4
3	Technical feasibility	6
4	Product requirements	17
5	MVP scope	22
6	Initial visual direction (temporary concept)	25
7	Privacy & security review	27
8	Performance & battery review	29
9	Agent reviews, issues & risks	30
10	Roadmap	37
11	Decisions requested & next steps	38
12	References	39

1 Executive summary & recommendation

PHASE 1 RECOMMENDATION

CONDITIONAL GO

FoldPulse is technically feasible with public Apple APIs, as an honest **hybrid tracker**. Two conditions must be met before significant design and build investment.

What we set out to learn

Can a third-party iOS app count how often an iPhone Duo is folded and unfolded, accurately, privately, and without unsupported APIs? And is the product worth building?

What we found

1. **iPhone Duo exposes a public hinge API (iOS 27.1).** `onHingeChange(isEnabled:_:)` in SwiftUI and `UIHingeInteraction` in UIKit report a hinge *status* (closed / partially open / fully open) and an *angle*. Apple documents both. **They only work while FoldPulse's UI is on screen.** There is no background mode.
2. **Background counting is possible only with the user's help.** iOS 27.2 added a Shortcuts automation trigger, "When my iPhone is opened or closed". FoldPulse can provide "Record Fold / Record Unfold" actions that run in the background. The user has to create the automation once. It is not yet verified whether it runs without a confirmation prompt and fires reliably while the phone is locked. **That is the biggest technical unknown.**
3. **Manual recording** and a widget button complete the picture. Core Motion and private APIs were examined and rejected.
4. **Widgets are well supported:** Home Screen, Lock Screen and StandBy (via the small widget). They refresh on a budget of roughly 40–70 updates a day, so they must show an "as of" time and never pretend to be live.
5. **The market already has fold counters.** *Duo Fold Counter* (about #10 in US Utilities), *FoldMeter* and *Foldometer* use the same mechanism. That proves the approach passes App Review. It also means FoldPulse must win on trust and polish. The team proposes a **tracking-coverage indicator** as the headline differentiator, with a **lean v1**.
6. **Local-only privacy is achievable**, and the "Data Not Collected" App Store label is justified, provided the seven privacy requirements in Chapter 7 are met.

What FoldPulse can honestly claim

> "FoldPulse counts your folds automatically while it's open, and in the background once you add the FoldPulse automation in Shortcuts. Every count shows where it came from."

FoldPulse must **not** claim to "automatically count every fold", and it must not estimate hinge wear.

Conditions of the GO

#	Condition	Owner
C-1	**Development access:** a Mac with Xcode 27.1+ and a physical iPhone Duo, or an equivalent macOS CI runner plus a device. The current cloud workspace runs Linux and cannot build or run iOS apps	Founder
C-2	**Device validation spike (Phase 1b, 1–2 days), run before Phase 2 design.** It must show that (a) FoldPulse receives the `closed` update when the phone is folded with the app open, and (b) the Shortcuts automation runs without a prompt and captures ≥ 95% of folds while locked, measured against an independent tally over ≥ 300 cycles	Architect + Founder
C-3	**Founder decisions** D-1 (compete despite existing apps), D-5 (business model) and D-6 (lean v1)	Founder

If C-2(b) fails, the team recommends **pausing** and re-evaluating: a foreground-only fold counter is unlikely to beat the existing competitors. **If C-2(a) fails**, automatic counting is limited to unfolds, and the scope is re-planned with the Founder.

2 Team, environment & method

Environment inspected

Item	Finding
Workspace	Linux cloud container; git repository `Church-Deacon-Election` (empty) on branch `claude/foldpulse-ios-development-gwxmj9`
iOS toolchain	**Not available** (no Xcode, Simulator or `swift` for iOS). Code cannot be compiled here
Research access	Apple developer documentation fetched directly (developer.apple.com JSON). Other sites accessed through web search summaries only (network policy)
Document tooling	Pandoc + headless Chromium (Playwright) for PDF; pdftoppm/pypdf for inspection
Agent capability	Claude Code subagents available

How the 12-agent team was run

Claude Code subagents were used for the **independent reviews**. Each reviewer ran as a separate agent, with its own context, its own research tools and no ability to edit the documents, so no agent approved its own work. Authoring roles were performed in the main session and are identified by role in every document.

Agent	Role	Phase 1 contribution	Mode
1	Chief Product Manager	Research questions, PRD, MVP scope, roadmap, issue log, consolidation	Main session
2	Founder Representative	Challenged scope, value, competition and retention	**Independent subagent**
3	Senior iOS Technical Architect	Hinge/Shortcuts/WidgetKit research, feasibility matrix, architecture direction	Main session
4	UI/UX Design Director	Initial visual direction, icon concepts, dashboard and widget sketches (Chapter 6)	Main session
5	Senior iOS UI Developer	No Phase 1 task (no code in Phase 1); reviewed the layout feasibility of the concepts	Main session
6	Senior iOS Software Engineer	Event pipeline concept (debounce, reconcile, single writer)	Main session
7	Data Analyst	Rewrote the metric and event rules	**Independent subagent**
8	Privacy & Security Engineer	Privacy review (Chapter 7)	**Independent subagent**
9	QA & Automation Engineer	Spike ground-truth protocol (≥ 300 cycles, independent tally)	Main session
10	Apple Platform Compliance Specialist	Independently re-verified APIs; App Review risks	**Independent subagent**
11	Performance & Battery Engineer	Battery and widget-budget review (Chapter 8)	Main session

Agent	Role	Phase 1 contribution	Mode
12	Independent Final Reviewer	Challenged all conclusions; GO/NO-GO assessment	**Independent subagent**

Initial research questions (Agent 1)

1. Does iPhone Duo expose hinge or fold state to third-party apps through public API? Since which iOS version?
2. Can fold events be observed in the foreground, in the background, and when the app is not running?
3. Which supported mechanisms (App Intents, Shortcuts, widgets, Controls) can extend tracking outside the app?
4. What can widgets show on the Home Screen, Lock Screen and in StandBy, and how fresh can they be?
5. Which metrics can be computed reliably given partial observation?
6. What does the local-only privacy promise require in practice (App Store label, privacy manifest, backups)?
7. Who else is solving this, and why would someone choose FoldPulse?

3 Technical feasibility

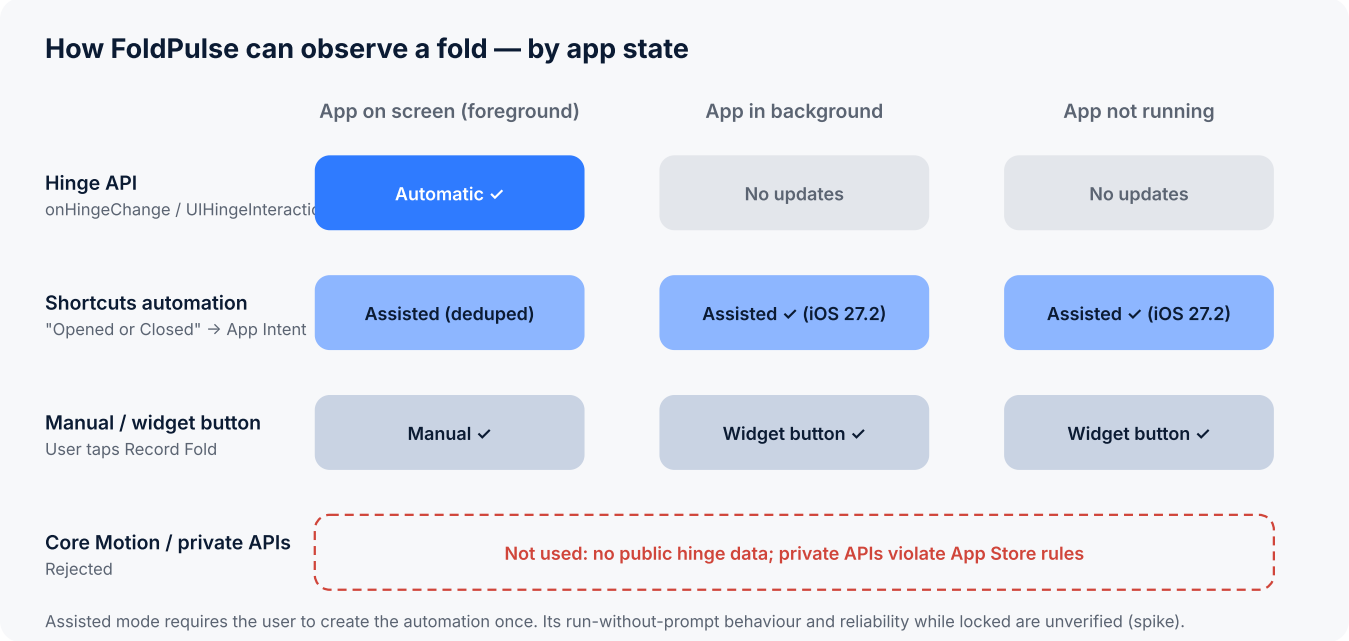


Figure 1 — Observation mechanisms by app state

1. Summary

Short answer: yes, FoldPulse can count folds on iPhone Duo using public Apple APIs. It can count every fold automatically only while the app is on screen. Counting while the app is closed depends on a user-configured Shortcuts automation.

The research found three public, documented mechanisms. Together they make a credible product:

1. **Hinge API (iOS 27.1 SDK)** — `UIHingeInteraction` (UIKit) and `onHingeChange` (SwiftUI) report a discrete hinge **status** (`closed`, `partiallyOpen`, `fullyOpen`, `unknown`) and a continuous **angle** in radians. These APIs are tied to a *view hierarchy*. They deliver updates only while FoldPulse's UI is live. They do **not** deliver events while the app is in the background or terminated.
2. **Shortcuts "Opened or Closed" automation trigger (iOS 27.2 system feature)** — Shortcuts can run a personal automation when iPhone Duo is opened, closed, or either. FoldPulse can expose App Intents ("Record Fold", "Record Unfold") that the automation calls. App Intents can run **in the background** (`supportedModes = [.background]`) without bringing the app forward. This is the only public route found to background counting. It requires a one-time setup by the user.
3. **Manual recording** — always available as a fallback and as a correction tool.

There is **no public API that lets a third-party app subscribe to hinge events in the background** without the user's Shortcuts automation. Nothing suggests one is coming. Core Motion, `UIDevice` and `BackgroundTasks` expose no hinge state.

Architect recommendation: CONDITIONAL GO. Build a *hybrid tracker*: automatic counting in the foreground, assisted counting in the background via Shortcuts, and manual recording. Label every event with its source.

> **Honesty constraint:** FoldPulse must not claim to "automatically count every fold." The accurate claim is: *"Counts folds automatically while open, and in the background once you add the FoldPulse automation in Shortcuts."*

2. Target device — iPhone Duo

Item	Finding	Source	Confidence
Device exists and is on sale	Announced 9 Sep 2026; on sale 23 Oct 2026	MacRumors roundup; Apple Newsroom (Oct 2026)	High
Displays	5.4" outer (cover) display, 7.6" inner display	MacRumors; Digital Trends	Medium (press, not Apple spec sheet)
Shipping OS	iOS 27.1. iOS 27.2 released 26 Oct 2026	MacRumors iOS 27.2 coverage	High
Content moves between displays when opening/closing	"As iPhone Duo is opened, closed, partially folded, and rotated, content transitions between displays"	Apple — *Preparing your app for iPhone Duo*	High (Apple primary)
Simulator support	Xcode 27.1 Device Hub can open, close, fold and rotate a simulated iPhone Duo	9to5Mac / Apple developer coverage	Medium

Implication: a foreground FoldPulse app keeps running as the device folds; its scene moves to the outer display. It can therefore observe the fold transition itself, not just the next unfold. This needs confirmation on a physical device (see §9, Validation spike).

3. API investigation

3.1 Hinge APIs — `UIHinge`, `UIHingeInteraction`, `onHingeChange`

Apple documentation (verified directly from developer.apple.com, 9 Nov 2026):

- `UIHingeInteraction` — "An interaction for observing the hinge state associated with the view's hierarchy." Available **iOS 27.1+**. Created with `init(updateHandler:)`; has `isEnabled`.
- "The interaction's handler is called when the hinge state changes, or when the interaction moves between hierarchies. When the interaction moves out of a hierarchy that provides hinge updates, the update's `hinge` is nil."
- `UIHinge` — "An object encapsulating the state of a single hinge." Properties: `angle` ("The current angle of the hinge, in radians") and `status`.
- `UIHinge.Status` — `closed` ("The hinge is closed"), `partiallyOpen`, `fullyOpen` ("open as far as the device allows"), `unknown`.
- UIKit's `Update` carries only `hinge`: no previous value and no timestamp. FoldPulse keeps its own last state and timestamps.
- On `angle`: "The rate and granularity of angle updates are system policy and can change based on system state, so don't depend on a particular update frequency or precision. If you only need to know whether the hinge is closed, partially open, or fully open, prefer `status` over the angle."

Apple Tech Talk 111464 (iPhone Duo): "SwiftUI provides a new onHingeChange modifier, and UIKit provides UIHingeInteraction. Both give you access to the high-level hinge status… Beyond the discrete status, it provides continuous updates of the hinge angle." The handler receives `(previous, current)` contexts, and "A null value indicates the app is running on a device without a hinge." Apple: "Hinge data is observed live, and is ideal for driving interactions or effects. For layout, use the arrangement and region APIs."

Architect's analysis:

- The API is attached to a **view in a live hierarchy**, not to the device or the process. No delegate, notification or background mode is documented. **Background delivery: not supported** (high confidence, by design).
- `status` is the right signal for counting: a fold is a transition into `.closed`, and an unfold is a transition out of `.closed`. The angle is useful only for optional visual effects, and it is not reliable enough for metrics (Apple explicitly declines to guarantee rate or precision; press reports roughly ±5° accuracy).
- The handler fires on *hierarchy moves* too. Moving between displays can therefore produce updates that are not physical hinge changes. Counting must be **edge-based** (state changed vs. last committed state) and **debounced**.
- `.unknown` must never be counted as a fold or unfold.

SwiftUI (verified in Apple docs after the Compliance review): `onHingeChange(isEnabled:_:)` — "Adds an action to perform when the hinge context of the view hierarchy changes." The closure receives `(DeviceHingeContext, DeviceHingeContext)` (old, new). `DeviceHingeContext.hinge` is an optional `DeviceHinge` with `angle` and `status`. `DeviceHinge.Status` is a **struct** with `closed`, `partiallyOpen` and `fullyOpen`. It has no `unknown`, so code must handle unrecognised values defensively.

Important nuance: Apple says the status is "determined by the system based on the current angle **and device orientation**". Rotating the phone could therefore change the status (most likely between `partiallyOpen` and `fullyOpen`) without any fold. FoldPulse counts only edges into and out of `closed`, which limits the exposure. The spike adds rotation-only tests to confirm that rotation never produces a `closed` edge.

3.2 Shortcuts "Opened or Closed" automation + App Intents

- iOS 27.2 adds a Shortcuts personal-automation trigger, "**When my iPhone is opened or closed**" (Display section; options: Opened / Closed / Either). It is exclusive to iPhone Duo. (9to5Mac, 29 Oct 2026; MacRumors, 25–26 Oct 2026.)
- Third-party apps cannot create automations themselves. The **user** creates the automation and picks an action. FoldPulse can provide **App Intents** (`RecordFoldIntent`, `RecordUnfoldIntent`) that appear as Shortcuts actions. It can also provide an **App Shortcut** that guides setup.
- App Intents can run without UI: Apple's `supportedModes` documentation says "Specify `background` to run the action entirely in the background."
- **Market evidence:** an existing App Store app, *Duo Fold Counter*, describes exactly this mechanism ("Count Fold"/"Count Unfold" actions on the Opened or Closed trigger, "even when the app isn't running"). That indicates the approach works and passes App Review. It is also a **competitive signal** (see PRD §4).

Open questions (require device validation):

Question	Why it matters
Can the automation run immediately with no confirmation banner/tap?	If it asks every time, background counting is impractical
Latency and reliability under Low Power Mode, Focus, locked device	Missed folds = wrong totals
Does it fire when the device is folded while locked or on the Lock Screen?	Many folds happen while locked

Question	Why it matters
Does it fire alongside the foreground hinge observer (double count)?	Requires de-duplication (see §6)
Is a timestamp passed, or must we use "now" at intent execution?	Accuracy of the event time

3.3 Other frameworks examined

Framework	Hinge/fold data?	Conclusion
Core Motion (`CMMotionManager`, device motion)	No hinge or posture API documented. Accelerometer/gyroscope only describe the whole device	Not usable. Inferring folds from motion would be unreliable and battery-heavy. Rejected
`UIDevice`	Orientation, battery, model — no hinge state documented	Not usable
`BackgroundTasks` (`BGAppRefreshTask`, `BGProcessingTask`)	Opportunistic, system-scheduled; no event triggers	Cannot capture events. May be used only for maintenance
SwiftUI reserved regions / `ArrangementView`	Geometry of the fold region for layout	For layout only; not an event source
SceneKit / ARKit	No hinge data	Not relevant
Private APIs / SpringBoard / IOKit	Possibly	**Forbidden** — App Store rejection, policy violation

3.4 Widgets (WidgetKit)

Verified from Apple's *Keeping a widget up to date*:

- Widgets are rendered by the system in a separate process. "Your widget extension is not continually active, even if the widget is onscreen."
- Budget: "For a widget the user frequently views, a daily budget typically includes from 40 to 70 refreshes… roughly… every 15 to 60 minutes."
- Reloads **don't count against the budget** when "the widget's containing app is in the foreground" or "the widget performs an app intent."
- "In StandBy, the system refreshes your widget's display at a system-defined rate that doesn't count against its budget."
- Timeline entries should be "at least about 5 minutes apart."

Families available on iOS: `systemSmall`, `systemMedium`, `systemLarge` (Home Screen); `accessoryCircular`, `accessoryRectangular`, `accessoryInline` (Lock Screen). `systemSmall` widgets appear in StandBy.

Implications:

- Each app-recorded event calls `WidgetCenter.reloadTimelines(ofKind:)`. That reload is free when the app is in the foreground and budgeted when it runs from a background intent. With heavy use (Duo owners may fold 50–150+ times a day), **background-intent reloads can exhaust the budget**. Widgets must therefore show an "as of" time and must never imply live data.
- The widget extension reads a small precomputed **snapshot** from the App Group container. It does not run analytics queries (widget memory limits are tight).

3.5 Lock Screen and StandBy

- Lock Screen accessory widgets: supported (circular gauge toward the next milestone, rectangular "Today · Lifetime", inline "38 folds today").
- StandBy shows Home Screen `systemSmall` widgets when an iPhone is charging, locked and on its side. **Unverified:** whether, and in which pose (closed or open), iPhone Duo enters StandBy. That is a device-validation item. FoldPulse supplies widgets that work well in StandBy. It does **not** and cannot replace the StandBy interface.

---

4. Feasibility matrix

Capability	Public API available?	Foreground	Background	Confidence	Evidence
Detect fold	**Yes** — hinge status transition to `.closed` (iOS 27.1)	**Yes** (automatic)	**Only via user-created Shortcuts automation** (iOS 27.2)	High (fg) / Medium (bg)	Apple UIHinge docs; Tech Talk 111464; 9to5Mac/MacRumors on the 27.2 trigger; Duo Fold Counter listing
Detect unfold	**Yes** — transition out of `.closed`	**Yes** (automatic)	**Only via Shortcuts automation**	High / Medium	Same
Read hinge angle	**Yes** — `UIHinge.angle` (radians)	Yes; rate/precision not guaranteed	**No**	High	Apple `angle` docs
Detect partially-open (tent/laptop) pose	Yes — `.partiallyOpen`	Yes	No	High	Apple `UIHinge.Status`
Update widgets	Yes — WidgetKit + App Group	Yes, unbudgeted while app foreground	Yes, budgeted (≈40–70/day)	High	Apple WidgetKit docs
Lock Screen widget	Yes — accessory families	n/a (system-rendered)	Shows last snapshot	High	Apple `WidgetFamily`
StandBy widget	Yes — `systemSmall` in StandBy	n/a	System-defined refresh rate	Medium (Duo-specific StandBy behaviour unverified)	Apple WidgetKit docs
Shortcuts / App Intents logging	Yes — App Intents with `.background` mode	Yes	Yes	Medium–High	Apple `supportedModes` docs
Interactive widget "+1" button	Yes — `Button(intent:)` in widget (iOS 17+). Runs **in the widget extension process** by default. **Inactive on a locked device**	n/a	Yes (unlocked only)	High	Apple *Adding interactivity to widgets*
Infer folds via Core Motion	No hinge API	—	—	High (rejected)	Apple Core Motion docs

5. Recommended tracking model

Sources (stored on every event, always shown in the UI):

Source	How	When	Trust
`hinge` (Automatic)	`onHingeChange` status edges	App on screen	Highest
`shortcut` (Assisted)	App Intent called by the Opened or Closed automation	App in background or not running	High (once validated)
`manual`	User taps Record Fold / Record Unfold, or a widget/Control button	Any time	User-asserted
`inferred`	Gap reconciliation. Fold and unfold must alternate, so **two consecutive events of the same type** (e.g. fold, fold) prove that one opposite event was missed. An `inferred` event of the missing type is created with an **uncertainty interval** `[previous event, next event]` rather than a precise time	On reconciliation (app activation, after import/edit)	Low. Shown in History with an explanation. Excluded from time-of-day, session and duration statistics. Excluded from headline totals by default (user toggle; Founder decision D-2)

Device-level observer. Exactly **one** hinge observer runs per process, attached to the active scene's root view. If the app shows windows on both displays, only the scene that is foreground-active may commit events. This prevents per-scene double counting.

`nil` hinge has two meanings. Apple: `hinge` is nil "when the interaction leaves a hierarchy that provides hinge updates", and Tech Talk 111464 says it is nil "on a device without a hinge". FoldPulse treats `nil` as **"no reading right now"**, never as "not an iPhone Duo". Whether the device is hinged is decided once, from the first non-nil reading ever received (persisted). Until then the app shows a neutral "Waiting for hinge..." state, not the manual-only mode.

6. Event correctness rules (revised after Data Analyst & Independent Reviewer review)

Definitions

- **Fold** = committed transition of hinge status from `.partiallyOpen` / `.fullyOpen` to `.closed`.
- **Unfold** = committed transition from `.closed` to `.partiallyOpen` or `.fullyOpen`.
- `.partiallyOpen ↔ .fullyOpen` transitions are **not** folds. They are not stored in the MVP.
- `.unknown` and `nil` never create events and never change the committed baseline.

Timing

- **Debounce:** a candidate status is committed once it has been stable for ≥ 250 ms (tuned in the spike). The event's `timestamp` is the moment the status was **first entered**, not the commit moment.
- **Flush on lifecycle change:** if the scene phase changes (for example, closing the phone sends the app to the background) while a candidate is pending, the candidate is committed immediately if it is not `.unknown`. This prevents a lost fold at the moment of closing.

Baseline on activation (prevents false events)

- The **first** reading after launch, after scene activation, or after a `nil` period only **sets the baseline**. It never creates a `hinge` event by itself.
- If that baseline contradicts the last stored event (e.g. last stored = fold, baseline = open), FoldPulse first tries to match a `shortcut` unfold that arrived in the meantime. If there is none, reconciliation records an `inferred` unfold covering the gap.

De-duplication (symmetric pairing)

- Two events are duplicates when they have the **same type**, there is **no opposite-type event between them**, and $|\Delta t| \leq$ **10 s**, in **either order** (a Shortcut may run before or after the debounced hinge commit, and it may lag while locked).
- Precedence when merging: `hinge` > `shortcut` > `manual` (including the widget button) > `inferred`. The kept event takes the timestamp of the highest-precedence source. The dropped event is retained as a hidden `duplicateOf` record for diagnostics and a "coverage" metric.
- Pairing is one-to-one: each event can absorb at most one duplicate per other source.

Derived metrics

- **Headline "Folds"** = fold events from `hinge` + `shortcut` + `manual` (+ `inferred` only if the user enables it). Shown as **"since install / since tracking began"**, never as the device's true lifetime.
- **Cycle / open session** = an unfold followed by the next fold, both non-inferred. A session is assigned to the local day it **started**. Sessions > 12 h are treated as missing data and excluded from duration statistics.
- **Coverage** (trust metric) = share of automatically captured folds this week (`hinge` + `shortcut`) out of all folds. Also the share of `shortcut` events whose automation self-test passed.
- **Averages** use **complete local days** from the first tracked day to **yesterday**, including zero days. The current partial day is excluded.

Time and time zones

- Store: `timestamp` (UTC instant), `timeZoneID` at capture and `localDate` (YYYY-MM-DD at capture). Daily buckets use the stored `localDate`; History sorts by UTC.
- Travel can make a local date appear twice or a day last 23 or 25 hours. Daily totals simply sum events by `localDate`. The time-of-day chart normalises each hour by the number of times that hour occurred, which handles the DST fall-back hour that happens twice.
- Weeks start on the user's locale `firstWeekday`.
- Editing an event's time keeps its original `timeZoneID`.

Edits, deletes and import

- Every event has `updatedAt`. Deletes leave a **tombstone** (`deletedAt`), so an import can't bring deleted events back.
- Import merges by `id`: the newer `updatedAt` wins, and tombstones win over older live copies.
- After any edit, delete or import, the reconciliation pass (alternation → dedupe → inferred → cycles) runs again for the affected range.
- **Milestones** store `reachedAt` the first time they are crossed and never fire again, even if later deletions drop the total below the threshold.

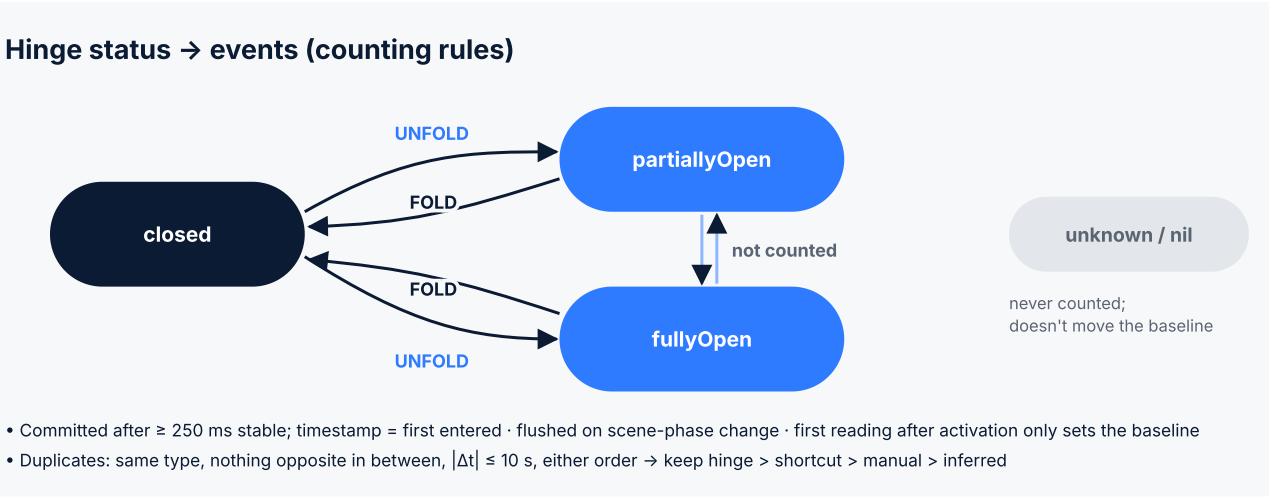


Figure 3 — Hinge status transitions and counting rules

7. Persistence and architecture direction

Decision	Recommendation	Rationale
Minimum OS	**iOS 27.1** (hinge APIs); the Shortcuts assisted mode requires the device to run 27.2+	iPhone Duo ships with 27.1; there is no reason to support older OSs
Device family	iPhone. Full features on iPhone Duo; other iPhones and iPads get a **manual-only mode with a clear notice**. `UIRequiredDeviceCapabilities` has **no hinge key**, so distribution cannot be limited to Duo. Guideline 2.4.1 expects iPhone apps to run on iPad, so test there too. **Opt out** of Apple-silicon Mac availability	App Review (2.4.1, 4.2, 4.3(b))
App Intents auth	`authenticationPolicy` left at the default `.alwaysAllowed`; never `requiresAuthentication`	Folds happen while locked
Persistence	**SwiftData** with the store in the **App Group** container	Mature by iOS 27, native, simple migrations via `VersionedSchema`; Core Data stays the fallback if spike issues appear
Stored data	Raw `FoldEvent` log is the **source of truth**; daily aggregates are computed and cached, never authoritative	Avoids drift; recomputation is cheap (100k events ≈ a few MB)
Widget data	App writes a compact `WidgetSnapshot` (JSON, < 4 KB) to the App Group after each commit	Widgets stay fast and within memory limits
Charts	Swift Charts	Native
Background intents	App Intents defined in the **app target** (the process launched in background), writing through the same persistence actor	Avoids two processes writing to SwiftData at once
Networking	**No networking code.** iOS has no network entitlement to turn off, so the promise is enforced by having no `URLSession`/networking imports, no third-party SDKs, a lint/CI check that fails on networking symbols, and proxy capture in Phase 5	Privacy promise
File protection	SwiftData store and snapshot use `NSFileProtectionCompleteUntilFirstUserAuthentication`	Background intents must be able to write while the device is **locked**. `Complete` protection would make locked-device writes fail

Decision	Recommendation	Rationale
Widget button writes	Apple confirms that the widget's "+ Fold" intent executes in the **widget extension process** by default. It must not open SwiftData itself; it appends to a small lock-free **pending-events queue file** in the App Group, which the app's single writer drains	Keeps one database writer (R-08)
Privacy manifest	Ship `PrivacyInfo.xcprivacy` declaring required-reason APIs (`UserDefaults` , file timestamp) and *no* tracking domains; App Store label "Data Not Collected"	Required for App Store submission

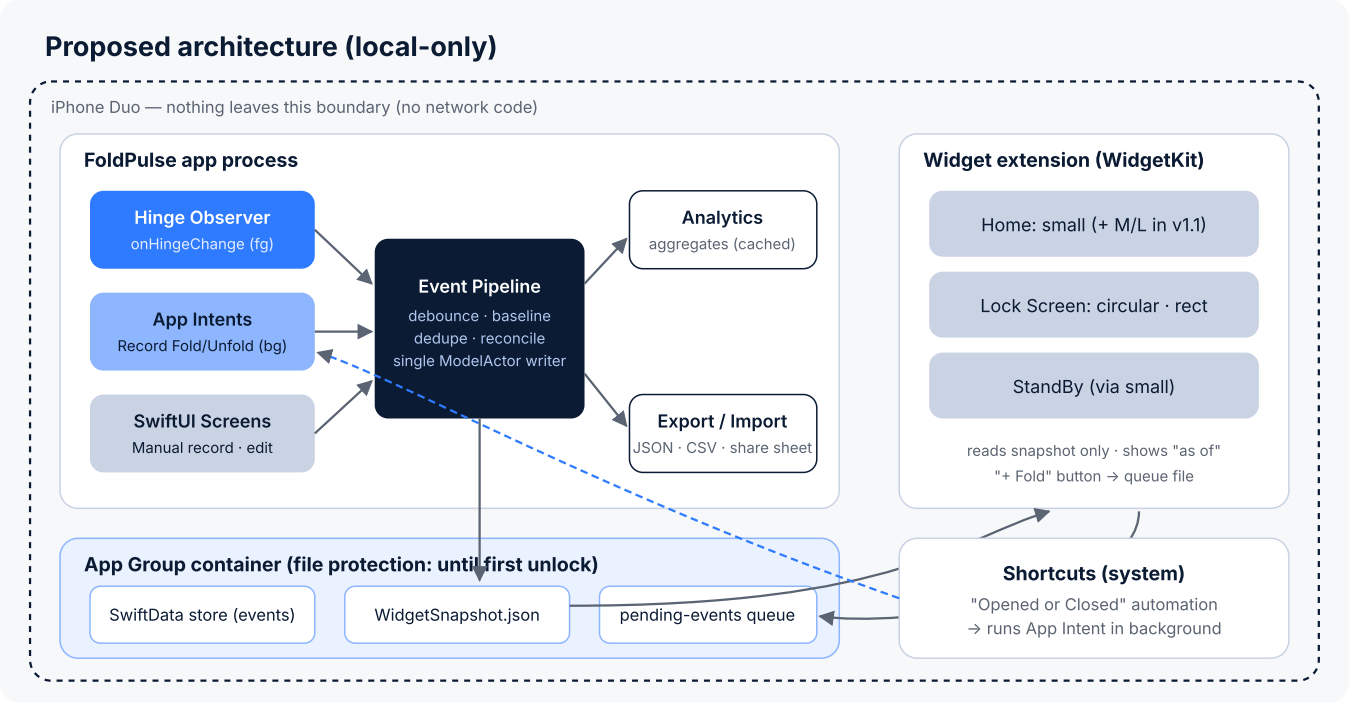


Figure 2 — Proposed local-only architecture

8. Constraints of the current development environment

- This project's cloud workspace runs **Linux. Xcode, the iOS Simulator and iOS builds are not available here.** Swift source, documentation and tests can be written, but the app cannot be **compiled, run or tested** in this environment.
- Phases 3–5 therefore need **one** of: (a) the Founder's Mac with Xcode 27.1+ and an iPhone Duo; (b) a macOS CI runner (e.g. GitHub Actions macOS) with Xcode 27.1 available; or (c) a Remote Control session to the Founder's Mac.
- No physical iPhone Duo is available to the team. **Every "device validation" item below is unverified until run on hardware.**

This is recorded as risk **R-01 (Critical)**.

9. Validation spike (proposed as **Phase 1b, before Phase 2 design**, ~1–2 days on a real device)

A throwaway diagnostic app (not shipped) that:

0. **Most important: confirm that FoldPulse stays foreground-active through a fold and receives the `.closed` update** (or learn what happens instead: scene moves to the outer display / app backgrounds / device locks). If the `.closed` update is not received, automatic *fold* counting in the foreground fails and only unfolds are automatic.
1. Logs every `onHingeChange` callback (status, angle, timestamp, which display, scene phase) while the device is folded/unfolded at different speeds.
2. Measures the false-transition rate under different debounce windows.
3. Adds a background App Intent and a user-created Opened/Closed automation, then checks: does it run immediately (no prompt)? Does it fire while locked? Under Low Power Mode? What is the latency?
4. Folds the device with the app in the foreground **and** the automation enabled → verifies de-duplication.
5. Checks StandBy entry on Duo (closed/open, charging, landscape) and widget presentation.
6. Measures battery impact over 24 h of the automation running.
7. **Rotation-only test:** rotate the device in each pose without folding, and confirm that no `closed` edge appears (status depends on orientation).
8. **Locked write test:** fold 50× while the phone is locked and confirm that background intents can write (file protection).

Ground-truth protocol: a tester performs **≥ 300 fold/unfold cycles** per condition (app foreground; app background, unlocked; locked; Low Power Mode). They keep an independent tally (a counter clicker), and the session is screen-recorded or video-recorded from a second device. Results are reported with a 95% confidence interval.

Exit criteria: foreground: ≤ 3 errors per 300 cycles (≥ 99%); background automation runs **without a confirmation prompt** and captures ≥ 95% of folds while locked; no duplicate after de-duplication.

Decision rule: if the background automation requires confirmation or misses > 5% of folds while locked, **stop and re-assess with the Founder** (a foreground-only product is unlikely to beat existing competitors). If the foreground `.closed` update is not delivered, automatic counting is limited to unfolds and the product is re-scoped.

---

10. Sources

- Apple — UIHingeInteraction: https://developer.apple.com/documentation/uikit/uihingeinteraction
- Apple — UIHinge: https://developer.apple.com/documentation/uikit/uihinge
- Apple — UIHinge.Status: https://developer.apple.com/documentation/uikit/uihinge/status-swift.enum
- Apple — UIHinge.angle: https://developer.apple.com/documentation/uikit/uihinge/angle
- Apple — Preparing your app for iPhone Duo: https://developer.apple.com/documentation/technologyoverviews/preparing-your-app-for-iphone-duo
- Apple Tech Talk 111464 (iPhone Duo hinge, scenes, accessories): https://developer.apple.com/videos/play/tech-talks/111464/
- Apple Developer Forums 847888 (fold posture / reserved regions): https://developer.apple.com/forums/thread/847888
- Apple — Keeping a widget up to date: https://developer.apple.com/documentation/widgetkit/keeping-a-widget-up-to-date
- Apple — WidgetFamily: https://developer.apple.com/documentation/widgetkit/widgetfamily
- Apple — AppIntent.supportedModes: https://developer.apple.com/documentation/appintents/appintent/supportedmodes

- Apple — SwiftUI onHingeChange(isEnabled:_:): https://developer.apple.com/documentation/swiftui/view/onhingechange(isenabled:_:)
- Apple — SwiftUI DeviceHinge / DeviceHinge.Status: https://developer.apple.com/documentation/swiftui/devicehinge
- Apple — Adding interactivity to widgets and Live Activities: https://developer.apple.com/documentation/widgetkit/adding-interactivity-to-widgets-and-live-activities
- Apple — AppIntent.authenticationPolicy: https://developer.apple.com/documentation/appintents/appintent/authenticationpolicy
- Apple — UIRequiredDeviceCapabilities: https://developer.apple.com/documentation/bundleresources/information-property-list/uirequireddevicecapabilities
- Apple — App Store Review Guidelines: https://developer.apple.com/app-store/review/guidelines/
- Apple — Core Motion CMMotionManager: https://developer.apple.com/documentation/coremotion/cmmotionmanager
- Apple Newsroom — Developers bring new app experiences to iPhone Duo (Oct 2026): https://www.apple.com/newsroom/2026/10/developers-bring-incredible-new-app-experiences-to-iphone-duo/
- MacRumors — iPhone Duo roundup: https://www.macrumors.com/roundup/iphone-duo/
- MacRumors — iPhone Duo automation when opened or closed (25 Oct 2026): https://www.macrumors.com/2026/10/25/iphone-duo-automation-opened-or-closed/
- MacRumors — iOS 27.2 features: https://www.macrumors.com/2026/10/26/ios-27-2-features/
- 9to5Mac — iOS 27.2 Shortcuts features (29 Oct 2026): https://9to5mac.com/2026/10/29/ios-27-2-gives-shortcuts-app-these-new-and-improved-features/
- 9to5Mac — iPhone Duo and non-optimized apps: https://9to5mac.com/2026/09/09/heres-how-iphone-duo-treats-apps-not-optimized-for-the-foldable-display/
- App Store — Duo Fold Counter (competitor): https://apps.apple.com/us/app/duo-fold-counter/id6827677447
- foldcounter.com — Fold Counter (competitor): https://foldcounter.com/

Research method note: Apple developer documentation was retrieved directly from developer.apple.com. Press sites (MacRumors, 9to5Mac, App Store pages) could not be fetched from this environment's network and were consulted through search-engine summaries. Those findings are marked Medium confidence where Apple has not confirmed them.

4 Product requirements

1. Problem statement

iPhone Duo is Apple's first foldable iPhone. Owners worry about, and are curious about, the hinge: *How often do I actually fold it? Am I wearing it out? Do I use the big screen or mostly the cover screen?* iOS has no built-in fold counter. Screen Time reports app usage, not hardware usage.

FoldPulse turns folding activity into a trustworthy, beautiful, private record.

2. Target users

Persona	Need	Priority
The Curious Early Adopter — bought iPhone Duo at launch, loves stats (Activity rings, Screen Time)	"Show me my numbers — lifetime, today, trends."	Primary
The Hinge Worrier — paid $1,999+, worries about durability, resale and warranty	"Give me a credible count since install, with its coverage, that I can export."	Primary
The Habit Tracker — wants to use the phone less or use the inner screen deliberately	"How often do I open it, and when?"	Secondary
The Tech Reviewer / Creator — makes durability content	"Export a dataset; show a widget on camera."	Secondary

Out of scope as users: owners of non-foldable iPhones (FoldPulse offers them nothing meaningful).

3. Product goals and success measures

Goal	Measure (FoldPulse contains no telemetry. Adoption and retention come from Apple's App Store Connect analytics, which are aggregated by Apple and cover only users who opted in to share with developers. No SDK is involved)
Accurate counts	Foreground automatic detection ≥ 99% and background assisted ≥ 95%, measured against an independent tally over ≥ 300 cycles (spike protocol)
Delight at a glance	Home/Lock Screen widget readable in < 1 s; dashboard first paint < 400 ms
Privacy	Zero network requests in a release build (verified by proxy capture in Phase 5)
Low effort	Background counting set up in ≤ 60 seconds via a guided flow
Battery	No measurable battery drain attributable to FoldPulse over 24 h (< 1% in Battery settings)

4. Competitive landscape (new finding — requires Founder attention)

The research found that **folding counters already exist** on the App Store for iPhone Duo:

- **Duo Fold Counter** — counts folds in real time while open; uses the iOS 27.2 "Opened or Closed" automation with "Count Fold / Count Unfold" Shortcuts actions; records missed changes as "inferred".
- **Fold Counter (foldcounter.com)** — positions itself as "hinge diagnostics".
- **FoldMeter – Fold Counter** and **Foldometer** — further App Store entrants.

- Open-source hobby projects (e.g. *iphone-duo-fold-counter* on GitHub).

Duo Fold Counter ranked about #10 in US Utilities (Similarweb, 2 Nov 2026) and already estimates "hinge life used". FoldPulse would arrive about 2–3 months later. **Polish alone is not a moat.** The team's view is that FoldPulse needs one trust feature the competitors lack (the coverage indicator below) and a lean, fast v1.

What this means: the core mechanism is proven and approvable. The market is not empty, though. FoldPulse must win on **experience, trust and polish** rather than on novelty:

1. **Tracking coverage indicator (headline differentiator)** — "Automatic capture covered 97% of your folds this week", plus a per-source breakdown (Automatic / Shortcut / Manual / Inferred). Users see how much to trust the number. 1a. **Factual context instead of fake "hinge health"** — e.g. "12,458 folds since install". FoldPulse cites a manufacturer rating only if Apple publishes one, and never estimates wear.
2. **Best-in-class widgets** — Home, Lock Screen and StandBy, with an honest "as of" freshness label.
3. **Analytics depth** — time-of-day patterns, open-session length, trends, milestones.
4. **Guided one-minute Shortcuts setup** with a self-test ("Fold your phone now — we'll confirm it worked").
5. **Strict privacy** — no network code at all, and a data export the user owns.

5. Scope summary

See `MVP_SCOPE.md` for the full in/out list. MVP pillars:

1. Hybrid tracking (Automatic in foreground, Assisted via Shortcuts, Manual).
2. Dashboard.
3. History with edit/delete and filters.
4. Analytics (day/week/month/year, averages, peaks, time of day).
5. Milestones (quiet celebration).
6. Widgets (Home small/medium/large, Lock Screen circular/rectangular/inline, StandBy via small).
7. Local data management (JSON/CSV export, JSON import, reset).

6. User stories and acceptance criteria

IDs map to MVP features. "AC" = acceptance criteria. All metrics follow `TECHNICAL_FEASIBILITY.md` §6.

Tracking

US-01 — Automatic counting while open. *As an owner, when FoldPulse is on screen and I fold or unfold my phone, I want it counted automatically.*

- AC1: A committed transition to `.closed` creates one `fold` event with source `hinge`.
- AC2: A committed transition out of `.closed` creates one `unfold` event with source `hinge`.
- AC3: `partiallyOpen ↔ fullyOpen` creates no fold/unfold event.
- AC4: `.unknown` never creates an event.
- AC5: Bouncing at the closing point (status flicker < debounce window) creates no extra events.
- AC6: The dashboard counter updates within 1 s of the event.

US-02 — Background counting via Shortcuts. *As an owner, I want folds counted even when FoldPulse isn't open.*

- AC1: FoldPulse provides "Record Fold" and "Record Unfold" Shortcuts actions that run without opening the app.
- AC2: A guided setup screen explains the steps, with illustrations, and deep-links to Shortcuts where possible.
- AC3: A self-test confirms that the automation fired ("We saw your fold at 14:02 ✓").
- AC4: Background events carry source `shortcut`.
- AC5: If a `hinge` event and a `shortcut` event of the same type occur within 3 s, exactly one event is kept.
- AC6: The app clearly states that this mode depends on iOS 27.2+ and on the user's automation.

US-03 — Manual recording. *As any user, I want to record a fold or unfold by hand.*

- AC1: Record Fold / Record Unfold buttons are on the Track screen; each creates an event with source `manual`.
- AC2: A 5-second Undo is offered after each manual entry.

US-04 — Correct mistakes. *As a user, I want to delete or edit an incorrect event.*

- AC1: Swipe-to-delete in History with confirmation; totals update immediately.
- AC2: Edit time, type and note. Editing a `hinge` / `shortcut` event marks it `edited` (the original source is preserved).

Dashboard

US-05 — At-a-glance numbers. Folds since install (hero; the label says "since install", not the device lifetime), today's folds, today's unfolds, 7-day daily average, last event (relative time + source), mini 7-day chart, progress to the next milestone.

- AC: All values are computed from the event log; with zero data, an empty state invites setup (no fake sample numbers, ever).

History

US-06 — Browse history. Chronological list grouped by day: time, type, source badge, note. Filter by date range and source.

- AC: Scrolls smoothly at 50,000 events (lazy loading).

Analytics

US-07 — Trends. Day/Week/Month/Year charts of folds; daily/weekly/monthly averages; highest-activity day; time-of-day heat strip.

- AC1: Aggregates use the event's recorded local time zone (see the Data Analyst's rules).
- AC2: Averages state their denominator ("per tracked day since 2 Nov").
- AC3: Open-session (unfold→fold) duration stats are shown only when ≥ 80% of the period's events are non-inferred, otherwise hidden with an explanation.

Milestones

US-08 — Celebrate. Milestones at 100, 500, 1k, 5k, 10k, 25k, 50k, 100k folds.

- AC: A one-time, dismissible celebration in the app; an optional local notification (off by default); no streaks, badges or gamified pressure.

Widgets

US-09 — Widgets. Home small/medium/large, Lock Screen circular/rectangular/inline; small works in StandBy.

- AC1: Every widget shows data as of its last snapshot and displays the "as of" time when the data is older than 15 minutes.
- AC2: Optional interactive "+ Fold" button on the Home Screen widget (source `manual`). Works only while unlocked (an Apple restriction); not available on Lock Screen widgets.
- AC3: Lock Screen widgets hide their counts when the device is locked and the user has chosen to hide sensitive data (`.privacySensitive()`).

Data

US-10 — Own my data. Export JSON (full fidelity) and CSV (formula-injection safe; a notice says exports reveal daily routine); import JSON (validate, preview, then merge without duplicates by event ID); reset everything with two-step confirmation.

- AC: An import of a corrupt or malicious file fails safely with a clear message and no partial writes.

Settings

US-12 — Tracking coverage. *As a Hinge Worrier, I want to know how complete my count is.*

- AC1: The dashboard shows weekly coverage = automatically captured folds ÷ all folds, plus the number of detected gaps (inferred).
- AC2: Tapping it explains how to raise coverage (set up or repair the automation).
- AC3: The Shortcuts self-test result and the date of the last automation-fired event are visible.

US-11 — Settings. Theme (System/Light/Dark), count inferred events (on/off), milestone notifications, Shortcuts setup and self-test, data management, privacy statement, About.

7. Non-functional requirements

- **Privacy:** no network calls, accounts, analytics, ads or cloud sync (see `PRIVACY_REVIEW.md`).
- **Accessibility:** Dynamic Type up to AX5, VoiceOver labels on all counters and charts (Audio Graphs for charts), Reduce Motion respected, contrast ≥ 4.5:1.
- **Localization-ready:** strings in a String Catalog; English at launch.
- **Performance:** cold start < 1 s; an event commit runs off the main thread; the widget snapshot write is < 10 ms.
- **Battery:** no continuous sensors, no location, no background audio tricks, no polling.

8. Explicit non-goals (MVP)

iCloud sync; Apple Watch app; social sharing/leaderboards; hinge "health" scores or warranty claims (FoldPulse cannot measure wear and must not imply it can); Android; monetization.

9. Open product decisions for the Founder

#	Decision	PM recommendation
D-1	Proceed despite existing competitors?	Yes — compete on quality, transparency and widgets

#	Decision	PM recommendation
D-2	Count `inferred` events in headline totals?	No by default; user toggle
D-3	Minimum OS iOS 27.1, iPhone Duo-focused?	Yes
D-4	Show a "Requires iPhone Duo" notice on other iPhones vs. a manual-only mode	Manual-only mode with a clear notice (reduces App Review risk)
D-5	Business model (free / paid up front / one-time unlock / tip jar)	**Decide before Phase 2**: it shapes design (paywall surfaces). Must stay privacy-preserving (StoreKit only, no ad or analytics SDKs)
D-6	Adopt the **lean v1** proposed by the Founder Representative (see MVP_SCOPE §4)?	Yes — ship faster against established competitors

5 MVP scope

1. MVP definition

> A privacy-first iPhone Duo app that **counts folds automatically while open, in the background through a guided Shortcuts automation, and manually**. It labels every event by source and presents the results on a dashboard, in history, in analytics, and in Home/Lock Screen/StandBy widgets.

2. In scope (MVP)

#	Feature	Priority	Feasibility	Notes
F1	Hinge tracking in the foreground (`onHingeChange`)	Must	Verified (API docs) / device spike pending	Edge-based, debounced
F2	App Intents: Record Fold / Record Unfold (background)	Must	Verified (API docs) / device spike pending	Powers the Shortcuts automation
F3	Guided Shortcuts setup + self-test	Must	Feasible	Biggest UX differentiator
F4	Manual record + undo	Must	Feasible	
F5	Dashboard	Must	Feasible	
F6	History list, filters, edit/delete	Must	Feasible	
F7	Analytics (D/W/M/Y, averages, peak day, time of day)	Must	Feasible	Swift Charts
F8	Milestones (in-app celebration)	Should	Feasible	Notification is optional, off by default
F9	Widgets: Home S/M/L, Lock Screen circular/rect/inline	Must (S, circular, rect) / Should (M, L, inline)	Feasible	StandBy through systemSmall
F10	Export JSON + CSV	Must	Feasible	Share sheet
F11	Import JSON	Should	Feasible	Validate → preview → merge
F12	Reset all data	Must	Feasible	Two-step confirm
F13	Settings (theme, inferred toggle, notifications)	Must	Feasible	
F14	Inferred-gap detection	Should	Feasible	Default excluded from totals
F15	**Tracking coverage indicator**	**Must**	Feasible	Added after Founder Rep review; key differentiator

3. Out of scope (MVP)

Item	Reason
Background hinge monitoring without Shortcuts	**Not possible** with public APIs
Hinge angle analytics (angle histograms)	Apple doesn't guarantee angle rate/precision; foreground only

Item	Reason
"Hinge health" / wear prediction	Not measurable; misleading; potential legal/marketing risk
iCloud / CloudKit sync	Violates the local-only principle (could be reconsidered later only with Founder approval)
Apple Watch, iPad, Mac	No hinge on those platforms; scope
Live Activities	Little value; adds complexity
Control Center control (iOS 18+ Controls)	Nice-to-have post-MVP (v1.1)
Monetization / IAP	Phase 6 decision
Localization beyond English	v1.1

4. Founder Representative challenge (Agent 2) — and resolution

Challenge	PM response	Outcome
"Competitors already do this. Why build it?"	The mechanism is commoditised; experience isn't. Differentiation comes from transparency, widgets, analytics and setup UX	**Escalated to Founder (D-1)**
"Large widget + inline + medium = 6 widget variants. Too many for v1."	Small + Lock circular + Lock rectangular are Must; the others are Should and can slip	Accepted
"Import JSON adds risk for few users."	Kept as Should. Device backups already restore App Group data, so import mainly helps after deleting the app or when moving data manually	Accepted as Should
"Inferred events will confuse people."	Default hidden from totals; shown only in History with an explanation	Accepted
"Milestone notifications need permission prompts — friction."	Off by default; the permission is requested only when the user turns them on	Accepted
"Analytics could be the whole app — keep v1 tight."	Limit to the list in F7; no custom ranges beyond the date filter	Accepted

4.1 Second-round challenge from the independent Founder Representative review

Challenge	PM response	Outcome
"Market risk understated: Duo Fold Counter is about #10 in US Utilities and already does background counting."	Agreed. Competitive section rewritten; risk R-21 added	Accepted
"14 features and 6 widgets is too big for a me-too launch. Cut the Large/Medium/inline widgets, the time-of-day heat strip, session durations and JSON import."	Partly agreed. Proposed **lean v1** below. JSON import is kept until we verify that App Group data is restored from an iCloud/Finder device backup; if it is, import moves to v1.1	**Founder decision D-6**
"Add a coverage indicator — the one trust feature competitors lack."	Agreed	Added as F15 (Must)
"If the automation needs confirmation or misses folds while locked, stop."	Agreed. This is now the spike decision rule	Adopted

Lean v1 (PM-recommended, pending D-6): F1–F6, F7 limited to D/W/M/Y charts, averages and peak day; F8; F9 limited to Home small + Lock Screen circular + rectangular (StandBy via small); F10; F12; F13; F14 (gaps shown only); F15. **Deferred to v1.1:** Medium/Large/inline widgets, time-of-day heat strip, session-duration

statistics, inferred-count toggle, JSON import (subject to the backup check).

5. MVP exit criteria

1. All Must features pass their acceptance criteria (PRD §6) on a physical iPhone Duo.
2. No Critical/High open bugs.
3. Zero network requests from a release build.
4. Spike exit criteria met (TECHNICAL_FEASIBILITY §9), or marketing claims revised.

6 Initial visual direction (temporary concept)

This chapter is the **UI/UX Design Director's initial direction**, offered so that the Founder can react early. It is a **temporary concept**: the brand, icon and screens are finalised in Phase 2. All numbers shown are illustrative, not user data.

Positioning: a premium, Apple-native instrument, closer to the Activity and Weather apps than to a game. The design is quiet and precise, with one accent colour. The interface's one moment of personality is the "pulse" line that traces each fold.



Figure 4 — Colour, type and icon concepts

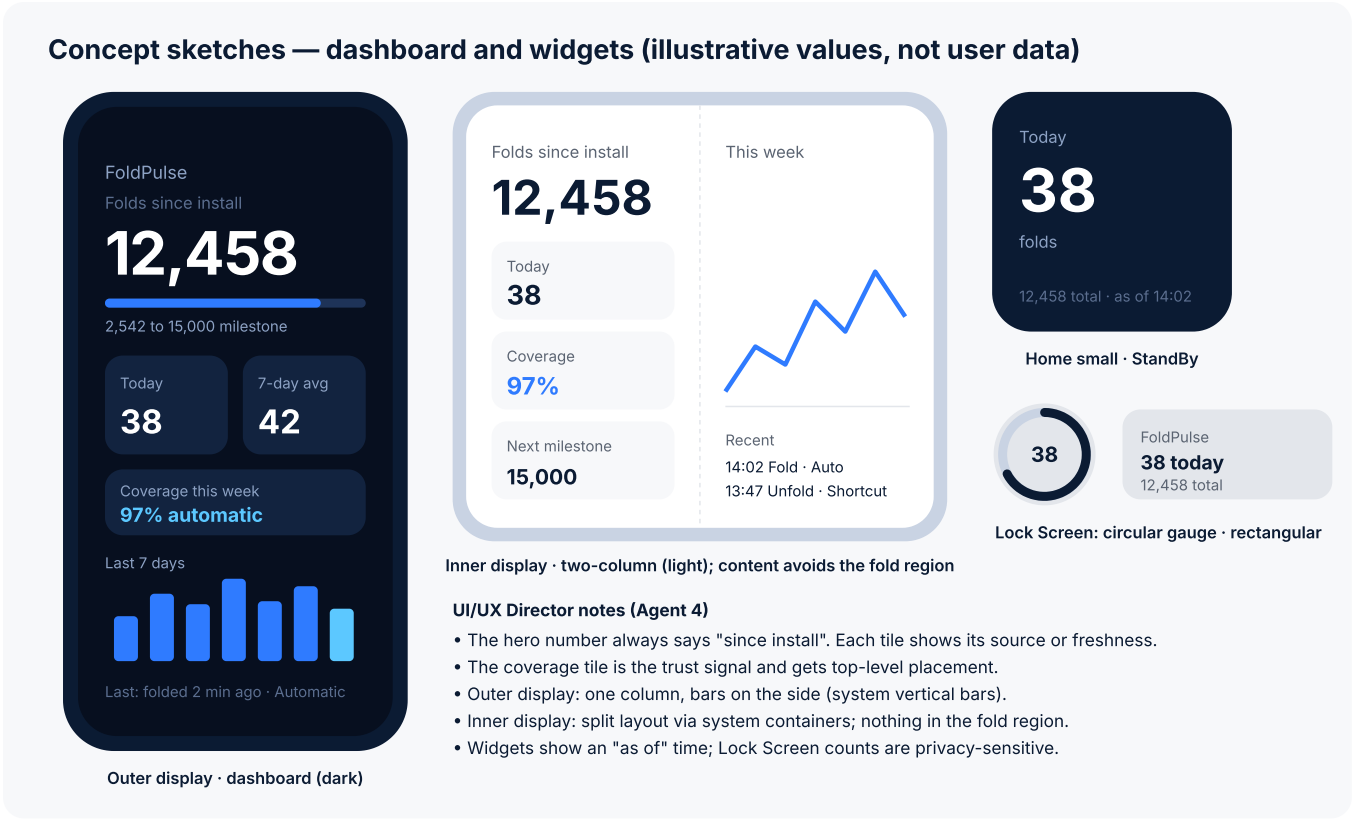


Figure 5 — Dashboard and widget concept sketches

Layout across displays. The outer (5.4") display shows a single-column dashboard, with the system presenting bars vertically at the side. The inner (7.6") display uses a two-column split built from system containers, and keeps content out of the fold region (reserved regions API). Both displays are designed from the start; neither is a scaled copy of the other.

Accessibility commitments for Phase 2: Dynamic Type to AX5; VoiceOver labels and Audio Graphs for charts; contrast ≥ 4.5:1 in both themes; Reduce Motion replaces the pulse animation with a fade; colour is never the only carrier of meaning (source badges use both icon and text).

7 Privacy & security review

1. Verdict

The proposed design **can honestly qualify for the App Store "Data Not Collected" label**. Apple defines *collect* as transmitting data off the device in a way that the developer or third parties can access. A file the user chooses to export through the share sheet is not collection. Seven requirements below must hold in the implementation.

2. Data inventory

Data	Where	Sensitivity	Leaves device?
Fold/unfold events (timestamp, time zone, type, source, note)	SwiftData store in the App Group container	**Medium.** Timestamps reveal sleep and daily routine	Only through a user-initiated export, or as part of the user's own iCloud/Finder device backup
Widget snapshot (today, total, next milestone, as-of)	App Group JSON	Low	Same as above
Settings	`UserDefaults` (App Group suite)	Low	Same
Notes (free text)	SwiftData	Potentially high (the user types it)	Same

No identifiers, location, contacts, health data, advertising ID or device fingerprinting.

3. Requirements

#	Requirement	Verification (Phase 5)
P-1	**No networking code.** No `URLSession` / `Network.framework` /WebKit fetches, no third-party SDKs, no analytics, ads, crash-reporting SDKs, CloudKit or remote database. (iOS has no "network entitlement" to remove, so this rule is enforced in the code itself)	CI check fails on networking symbols; proxy capture of a release build shows zero requests
P-2	**Privacy manifest** (`PrivacyInfo.xcprivacy`) in both the app and the widget extension. `NSPrivacyTracking = false` , no tracking domains, collected data types = none. Required-reason APIs as used, likely `UserDefaults` (`1C8F.1` for App Group access, `CA92.1` app-only), file timestamps (`C617.1`), and system boot time (`35F9.1`) if monotonic clocks are used for debounce	Xcode privacy report
P-3	**File protection** `CompleteUntilFirstUserAuthentication` for the store, snapshot and queue (needed for locked-device writes); never weaker	Spike locked-write test
P-4	**Export safety:** CSV cells starting with `=` , `+` , `-` or `@` are escaped (formula injection). Before sharing, a notice says the file reveals daily routine. Exports go to a temporary file deleted after sharing	Unit tests
P-5	**Import safety:** ≤ 20 MB, strict schema decode, field bounds (dates 2026–2100, notes ≤ 500 chars), transactional merge, preview before commit, nothing executed or rendered as HTML	Fuzz tests with corrupt and oversized files
P-6	**Lock Screen widgets** use `.privacySensitive()` for counts	UI review

#	Requirement	Verification (Phase 5)
P-7	**Disclosure:** the in-app privacy statement and the App Store policy say "Data stays on your iPhone. It is included in your own device backups (iCloud/Finder) if you use them. FoldPulse never sends it anywhere." A **privacy policy URL** is required (guideline 5.1.1(i))	Phase 6 checklist

4. Other notes

- **Notifications:** local `UNUserNotificationCenter` only. No push entitlement and no APNs.
- **App Intents:** `authenticationPolicy` stays at the default `.alwaysAllowed` so that folds can be recorded while the phone is locked. The intents only *append* an event. They expose no data on the Lock Screen and accept no parameters except an optional note.
- **Unverified callers:** any user shortcut can call "Record Fold". This is a data-quality issue, not a privacy one; see risk R-25.
- **Reset:** deletes the store, snapshot, queue and settings, then reloads widgets.
- **Monetisation:** any future model must use StoreKit only. Ad or analytics SDKs are ruled out.

8 Performance & battery review

Owner: Agent 11 — Performance and Battery Engineer

Area	Assessment	Design rule
Hinge observation	Event-driven system callbacks, no polling. Negligible cost while the app is on screen	Observer active only in the foreground; angle ignored except for optional effects
Background intents	Each automation run launches FoldPulse briefly in the background. About 100–300 runs a day for heavy users	Intent work kept minimal: append one event, update the snapshot, coalesce widget reloads. Target < 50 ms CPU per run
Widget refresh	Budget of about 40–70 reloads a day (Apple). Reloads from the foreground app and from widget intents are free	Reload immediately when in the foreground. In the background, at most one reload per 5 minutes; widgets show "as of"
Storage	100,000 events ≈ a few MB	Raw event log + cached daily aggregates, invalidated on edit
Startup	Dashboard from the cached snapshot first, then recompute	Cold start < 1 s; first paint < 400 ms
Prohibited	Continuous Core Motion, location, background audio, silent push, timers	Never used

Open item: the real battery cost of hundreds of daily automation launches is unmeasured. It is spike step 6 (24-hour test).

9 Agent reviews, issues & risks

Each reviewer below ran as an independent subagent. Its findings and the response to them are summarised here; the full trail is in the issue log (Chapter 9).

Apple Platform Compliance Specialist (Agent 10)

Finding	Severity	Response
Confirmed the hinge APIs, `supportedModes(.background)` (iOS 26+) and the WidgetKit budget wording against Apple's docs	—	Agreed
SwiftUI `onHingeChange(isEnabled:_:)` is documented; the first research pass missed it (wrong URL). `DeviceHinge.Status` is a struct and depends on angle *and orientation*	High	Verified independently by the Architect; feasibility report corrected; R-15 closed, R-22 added; rotation test added to the spike
Widget button intents run in the **widget extension process** and are **inactive while locked**	High	Verified in Apple docs; pending-events queue design; button is unlocked-only
No hinge key in `UIRequiredDeviceCapabilities` ; iPhone apps must run on iPad (2.4.1)	Medium	Manual-only mode with a notice; iPad testing; opt out of Mac
Saturated category (4.3(b)); privacy policy URL required (5.1.1(i))	Medium	R-23, R-24; App Review notes with demo video

Privacy & Security Engineer (Agent 8)

Finding	Severity	Response
"Data Not Collected" label is valid; local notifications only is correct	—	Agreed
Privacy manifest (`PrivacyInfo.xcprivacy`) missing from the plan, for app and widget	High	Requirement P-2
Locked-device writes fail under `Complete` file protection	High	`CompleteUntilFirstUserAuthentication` (P-3); spike test
App Group data is included in iCloud/Finder backups, so "no cloud" wording overstated	Medium	Disclosure P-7; R-11 downgraded
CSV formula injection; exports reveal daily routine	Medium	**P-4**
`.privacySensitive()` on Lock Screen; shortcut events are unverified	Low	P-6; R-25

Founder Representative (Agent 2)

Finding	Severity	Response
Market risk understated: Duo Fold Counter is about #10 in US Utilities and already does background counting and a "hinge life" estimate	High	PRD §4 rewritten; R-21
MVP too big for a late entrant	High	Lean v1 proposed; **Founder decision D-6**

Finding	Severity	Response
Add a tracking-coverage indicator as the trust differentiator	High	Added as F15 / US-12 (Must)
Success can't be measured with local-only metrics	Medium	Use App Store Connect analytics (opt-in, Apple-aggregated; no SDK)
Users want a "hinge life" number	Medium	Provide factual context ("folds since install"); still no wear estimate. **Disagreement noted:** the Founder Rep considers this a retention risk; the PM and Compliance consider a wear estimate misleading
Recommendation: Conditional GO on a slimmed v1; stop if the automation needs confirmation or misses folds while locked	—	Adopted as the decision rule

Data Analyst (Agent 7)

Finding	Severity	Response
Inferred-event rule was self-contradictory and had no time for day bucketing	Critical	Rewritten: alternation-based, uncertainty interval, bucketed on the later bound
First reading after resume would create a false unfold and double count	Critical	Baseline-on-activation rule
3-second time-based dedupe was order-dependent	High	Symmetric one-to-one pairing, ±10 s, nothing opposite in between
Debounce timing and lost pending state on backgrounding	High	Timestamp = first entered; flush on scene-phase change
Import resurrects deleted events	High	Tombstones + `updatedAt`; re-run reconciliation
Averages, DST, sessions, milestones, widget at midnight	Medium/Low	All adopted (feasibility §6)

Independent Final Reviewer (Agent 12)

Finding	Severity	Response
Verified `UIHingeInteraction`, `UIHinge.Status`, `supportedModes`, the widget budget. UIKit `Update` has no previous value	—	Agreed; noted in the report
The core assumption, that a foreground fold is observable, is tested too late	High	Spike moved to **Phase 1b, before design** (condition C-2)
Phase 1 was marked "Complete" before review; the PDF was referenced before it existed	High	Statuses corrected; this PDF produced and checked
`nil` hinge has two meanings	Medium	Hinged-device decision persisted from the first non-nil reading
Locked writes; second writer from the widget; one-directional dedupe; scene duplication	Medium	All addressed in the design
Accuracy targets statistically weak	Medium	≥ 300 cycles with independent tally, 95% CI
"Lifetime"/"trustworthy" overclaims	Medium	"Since install" wording

Finding	Severity	Response
No "network entitlement" exists; privacy manifest missing	Low	Corrected
"Mitigated" statuses overstate progress	Low	Relabelled "Designed – unverified"
Docs live in an unrelated repository	Low	Raised with the Founder (I-17)
Recommendation: CONDITIONAL GO (R-01, D-1, spike before Phase 2)	—	Adopted as the consolidated recommendation

Disagreements recorded

1. **Hinge-life number** — Founder Rep for a factual substitute; PM and Compliance against any wear estimate. Resolution proposed: factual counts only. **Founder may decide.**
2. **Lean v1 cuts** — Founder Rep would also cut JSON import; PM would keep it until the device-backup restore behaviour is verified.
3. **Spike timing** — Independent Reviewer requires it before Phase 2; PM notes Phase 2 brand work (not screens) could run in parallel. Consolidated recommendation: **spike first**.

Risk register & issue log

Severity: Critical · High · Medium · Low. Likelihood: H / M / L.

1. Risk register

ID	Risk	Sev.	Lik.	Owner	Mitigation	Status
R-01	**No macOS/Xcode or physical iPhone Duo is available to the team.** The cloud workspace runs Linux and cannot build or run iOS apps	Critical	H	Architect	Founder provides a Mac with Xcode 27.1+ and an iPhone Duo (or a macOS CI runner / Remote Control to the Founder's Mac) before Phase 3. All device claims stay "unverified" until then	**Open — Founder action**
R-02	The Shortcuts automation asks for confirmation each run, or doesn't fire while locked → background counting impractical	High	M	Architect / Compliance	Validation spike step 3. Fallback: foreground + manual; revise marketing	Open
R-03	Hinge callbacks fire on display/hierarchy moves → double counts	High	M	Engineer / Data Analyst	Edge-based commit vs. last committed state; 250 ms debounce; dedupe window	Designed – unverified
R-04	Foreground hinge event + Shortcut event for the same physical fold → double count	High	H	Data Analyst	3 s cross-source dedupe; keep `hinge`	Designed – unverified
R-05	Established competitors (Duo Fold Counter, Fold Counter) reduce the market opportunity	High	H	PM / Founder Rep	Differentiate on transparency, widgets, setup UX, privacy. **Founder decision D-1**	Open — Founder decision

ID	Risk	Sev.	Lik.	Owner	Mitigation	Status
R-06	App Review: app is mainly useful on iPhone Duo, and distribution can't be restricted (no hinge key in `UIRequiredDeviceCapabilities`; iPhone apps must run on iPad, 2.4.1)	Medium	M	Compliance	Manual-only mode with a notice on non-hinge devices; test on iPad; opt out of Mac	Open
R-07	Widget reload budget exhausted by frequent background intent reloads → stale widgets	Medium	H	Performance	"As of" label; coalesce reloads (≥ 5 min apart from background); unbudgeted refresh when the app is foreground	Designed – unverified
R-08	SwiftData + App Group + background intent concurrency issues	Medium	M	Architect	Single `ModelActor` writer in the app process; intents in the app target; widget reads only a JSON snapshot. Fallback Core Data	Open — verify in Phase 3
R-09	Users read FoldPulse as a measure of hinge wear/warranty	Medium	M	PM / Compliance	Copy review; disclaimer; no "health" scores	Designed – unverified
R-10	Time-zone / DST errors in daily aggregates	Medium	M	Data Analyst	Store UTC instant + capture time-zone ID; bucket by `Calendar` in the capture zone; DST tests	Designed – unverified
R-11	Data loss on delete/reinstall. Note: App Group data **is** included in iCloud/Finder device backups, so a restore from backup recovers it. Deleting the app does not	Low	M	Engineer / Privacy	Prominent export; disclose backup behaviour	Designed – unverified
R-12	Malicious/corrupt JSON import	Medium	L	Privacy	Size cap (≤ 20 MB), schema validation, transactional import, no code paths that evaluate content	Designed – unverified
R-13	Press-reported facts (iOS 27.2 trigger, StandBy on Duo, ±5° angle accuracy) are wrong or change	Medium	M	Compliance	Verify on the device during the spike; Apple primary docs take precedence	Open
R-14	iPhone Duo StandBy behaviour unknown (pose / whether supported)	Low	M	Compliance	Spike step 5; widgets work regardless	Open
R-15	SwiftUI `onHingeChange` signature unconfirmed	Low	L	Architect	**Resolved:** documented as `onHingeChange(isEnabled:_:)` with `DeviceHingeContext`	Closed
R-22	Hinge status depends on **device orientation** as well as angle, so rotation could create spurious status changes	Medium	M	Architect / Data Analyst	Count only `closed` edges; rotation-only spike test	Designed – unverified
R-23	App Review 4.3(b) (saturated category) / 2.3.1(a) (undisclosed features)	Medium	M	Compliance	Explain the differentiators in the review notes, with a demo video of the Shortcuts setup	Open

ID	Risk	Sev.	Lik.	Owner	Mitigation	Status
R-24	Privacy policy URL required (5.1.1(i)) even when no data is collected	Medium	H	Compliance	Host a simple static privacy policy (Phase 6)	Open
R-25	Any shortcut can call Record Fold, so `shortcut` events are not proof of a physical fold	Low	M	Privacy / Data Analyst	Label them "Shortcut"; the self-test verifies the automation; coverage metric	Designed – unverified
R-17	Fold while locked: background intent cannot write if the store uses `Complete` file protection	Medium	M	Privacy / Architect	`CompleteUntilFirstUserAuthentication`; test in spike	Designed – unverified
R-18	Widget "+ Fold" intent runs in the extension process → second database writer	Medium	M	Architect	Pending-events queue file drained by the app	Designed – unverified
R-19	**Foreground fold may not be observable** if closing the phone backgrounds or locks the app before the `.closed` update arrives	High	M	Architect	Spike step 0 runs **before Phase 2**; re-scope if it fails	Open
R-20	Overstating the count as "lifetime" or "trustworthy". The count starts at install and can be edited	Medium	M	PM / Compliance	Copy uses "since install"; source breakdown and coverage metric	Designed – unverified
R-21	Late entry. Competitors (Duo Fold Counter ~#10 US Utilities on 2 Nov 2026; FoldMeter; Foldometer) are 2–3 months ahead	High	H	Founder Rep	Slim v1, ship fast, lead with the coverage/trust feature	Open — Founder decision
R-16	Scope creep (Watch, sync, social)	Low	M	PM	MVP_SCOPE out-of-scope list; change-control via Founder	Controlled

2. Issue log (Phase 1 agent reviews)

ID	Description	Sev.	Raised by	Responsible	Proposed resolution	Status	Final decision
I-01	The original brief's assumption that FoldPulse can "automatically count every fold" is false for background use	Critical	Architect	PM	Hybrid model; honest claim wording	Resolved	Adopted in PRD/feasibility
I-02	No ability to compile or device-test in the current environment	Critical	Architect	Founder	See R-01	Open	**Awaiting Founder**
I-03	Competitors already exist	High	Founder Rep	PM	Differentiation plan PRD §4	Open	**Awaiting Founder (D-1)**
I-04	Should inferred events count toward totals?	Medium	Data Analyst	PM	Excluded by default; toggle	Proposed	Awaiting Founder (D-2)

ID	Description	Sev.	Raised by	Responsible	Proposed resolution	Status	Final decision
I-05	Storing DailyStatistics vs. computing them	Medium	Data Analyst	Architect	Compute from the event log; optional cache invalidated on edit	Resolved	Event log is authoritative
I-06	"Cycle" definition was ambiguous in the brief	Medium	Data Analyst	Data Analyst	Unfold→next fold, both non-inferred	Resolved	Adopted
I-07	Milestone notifications require a permission prompt	Low	Founder Rep	UX	Opt-in only	Resolved	Adopted
I-08	Widget count (6 variants) is heavy for v1	Low	Founder Rep	PM	Tiered Must/Should	Resolved	Adopted
I-09	Press sources could not be fetched directly (network policy); relied on search summaries	Medium	Independent Reviewer	Architect	Confidence downgraded to Medium; device verification	Resolved (documented)	Accepted
I-10	Local notifications for milestones are local only — confirm that no APNs entitlement is needed	Low	Privacy	Privacy	Use `UNUserNotificationCenter` local only; no push entitlement	Resolved	Adopted
I-11	SwiftUI hinge API was missed in the first research pass (wrong URL)	Medium	Compliance	Architect	Verified and documented	Resolved	Adopted
I-12	Widget button intent runs in the extension process; inactive while locked	High	Compliance	Architect	Queue file; widget button is unlocked-only and Should, not Must	Resolved in design	Adopted
I-13	Missing privacy manifest, privacy policy URL, iCloud Backup disclosure, CSV injection, export sensitivity	High	Privacy	Privacy	See PRIVACY_REVIEW.md	Resolved in design	Adopted

ID	Description	Sev.	Raised by	Responsible	Proposed resolution	Status	Final decision
I-14	Inferred-event rule contradicted itself; resume could create a fake unfold; dedupe was one-directional	Critical	Data Analyst / Independent Reviewer	Data Analyst	Rules rewritten (feasibility §5–§6)	Resolved	Adopted
I-15	Phase 1 was marked "Complete" before the reviews happened	High	Independent Reviewer	PM	Status → "pending Founder review"	Resolved	Adopted
I-16	Core assumption (foreground fold observable) is tested too late	High	Independent Reviewer	Architect	Spike moved to Phase 1b, before design	Proposed	Awaiting Founder
I-17	Project docs live in a repository named "Church-Deacon-Election"	Low	Independent Reviewer	Founder	Move to a dedicated FoldPulse repository before Phase 3	Open	Awaiting Founder

10 Roadmap

Every phase ends with a PDF report and an explicit Founder approval gate.

Phase	Objective	Key outputs	Prerequisites	Status
1 — Research & Feasibility	Verify what's possible; define the MVP	Feasibility report, PRD, MVP scope, risks, roadmap, `phase-01-feasibility.pdf`	—	**Deliverables ready — pending Founder review**
1b — Device validation spike (recommended)	Prove on a real iPhone Duo: foreground fold observed; Shortcuts automation runs while locked without prompt	Spike report appended to the feasibility report	Founder's Mac with Xcode 27.1 + iPhone Duo (R-01)	Proposed
2 — Brand, UI/UX & Design	Visual identity, icon, screens, widgets, accessibility	Design system, screen specs, flows, widget design, mockups, `phase-02-design.pdf`	Phase 1 approval	Not started
3 — Architecture & Foundation	Xcode project, data layer, design system in code, tests	Architecture, decisions, DB design, buildable project, `phase-03-architecture.pdf`	Phase 2 approval; **Mac + Xcode 27.1 + iPhone Duo access (R-01)**	Not started
4 — MVP Development	Implement F1–F14 incrementally	Source, dev log, test results, `phase-04-development.pdf`	Phase 3 approval	Not started
5 — QA & Validation	Functional, data, widget, privacy, performance testing	Test plan/results, bugs, reviews, `phase-05-testing.pdf`	Phase 4 approval	Not started
6 — Release Preparation	App Store assets, privacy disclosure, final build	Store copy, checklist, disclosure, `phase-06-release.pdf`	Phase 5 approval	Not started

Post-MVP candidates (not committed)

Version	Candidates
v1.1	Control Center control ("Record Fold"), localization, posture-time analytics (partially-open "tent" time) if foreground data proves useful
v1.2	Optional user-initiated encrypted backup file; yearly "Fold Wrapped" summary (on device)
Later (requires Founder privacy review)	Optional iCloud sync; Apple Watch glance

11 Decisions requested & next steps

Please reply with your decisions. Defaults shown are the team's recommendations.

#	Decision needed	Team recommendation
A	**Approve Phase 1** (this report and the docs)	Approve
D-1	Build FoldPulse despite existing fold-counter apps?	Yes — compete on trust (coverage), widgets and polish
D-2	Count `inferred` (gap) events in headline totals?	No by default
D-3	Minimum iOS 27.1, iPhone Duo-focused	Yes
D-4	Non-Duo devices: manual-only mode with a notice	Yes
D-5	Business model: free / paid up front / one-time unlock / tip jar	Decide before Phase 2 (no ads, no analytics SDKs)
D-6	Adopt the lean v1 (MVP_SCOPE §4.1)	Yes
C-1	Provide a Mac with Xcode 27.1+ and an iPhone Duo (or macOS CI + device)	Required before the spike
C-2	Run the Phase 1b device spike before Phase 2	Yes
I-17	Move FoldPulse into its own repository	Yes, before Phase 3

Next phase

If approved, the next step is the **Phase 1b validation spike** (with your device). After it, **Phase 2 — Brand, UI/UX & Design** produces the design system, the final icon, screen specifications, user flows, widget designs, mockups and `phase-02-design.pdf`. No application code is written until Phase 3 is approved.

12 References

Apple primary sources were retrieved directly from developer.apple.com on 9 November 2026. Press and App Store sources were consulted through search-engine results, because those sites are blocked by this environment's network policy, and are rated Medium confidence.

Apple — primary

- UIHingeInteraction — https://developer.apple.com/documentation/uikit/uihingeinteraction
- UIHinge / UIHinge.Status / angle — https://developer.apple.com/documentation/uikit/uihinge
- SwiftUI onHingeChange(isEnabled:_:) — https://developer.apple.com/documentation/swiftui/view/onhingechange(isenabled:_:)
- SwiftUI DeviceHinge — https://developer.apple.com/documentation/swiftui/devicehinge
- Preparing your app for iPhone Duo — https://developer.apple.com/documentation/technologyoverviews/preparing-your-app-for-iphone-duo
- Tech Talk 111464 (iPhone Duo) — https://developer.apple.com/videos/play/tech-talks/111464/
- Developer Forums 847888 — https://developer.apple.com/forums/thread/847888
- Keeping a widget up to date — https://developer.apple.com/documentation/widgetkit/keeping-a-widget-up-to-date
- WidgetFamily — https://developer.apple.com/documentation/widgetkit/widgetfamily
- Adding interactivity to widgets — https://developer.apple.com/documentation/widgetkit/adding-interactivity-to-widgets-and-live-activities
- AppIntent.supportedModes — https://developer.apple.com/documentation/appintents/appintent/supportedmodes
- AppIntent.authenticationPolicy — https://developer.apple.com/documentation/appintents/appintent/authenticationpolicy
- UIRequiredDeviceCapabilities — https://developer.apple.com/documentation/bundleresources/information-property-list/uirequireddevicecapabilities
- Privacy manifest required-reason APIs — https://developer.apple.com/documentation/bundleresources/app-privacy-configuration/nsprivacyaccessedapitypes/nsprivacyaccessedapitypereasons
- App privacy details — https://developer.apple.com/app-store/app-privacy-details/
- App Store Review Guidelines — https://developer.apple.com/app-store/review/guidelines/
- Core Motion CMMotionManager — https://developer.apple.com/documentation/coremotion/cmmotionmanager
- Apple Newsroom, iPhone Duo developer apps (Oct 2026) — https://www.apple.com/newsroom/2026/10/developers-bring-incredible-new-app-experiences-to-iphone-duo/

Press and market — secondary

- MacRumors iPhone Duo roundup — https://www.macrumors.com/roundup/iphone-duo/
- MacRumors, Opened-or-Closed automation (25 Oct 2026) — https://www.macrumors.com/2026/10/25/iphone-duo-automation-opened-or-closed/
- MacRumors, iOS 27.2 features — https://www.macrumors.com/2026/10/26/ios-27-2-features/
- 9to5Mac, iOS 27.2 Shortcuts features — https://9to5mac.com/2026/10/29/ios-27-2-gives-shortcuts-app-these-new-and-improved-features/

- 9to5Mac, non-optimised apps on iPhone Duo — https://9to5mac.com/2026/09/09/heres-how-iphone-duo-treats-apps-not-optimized-for-the-foldable-display/
- Digital Trends, iPhone Duo — https://www.digitaltrends.com/phones/iphone-fold-everything-we-know-about-apples-ultra-foldable/
- App Store, Duo Fold Counter — https://apps.apple.com/us/app/duo-fold-counter/id6827677447
- Similarweb, Duo Fold Counter ranking — https://www.similarweb.com/app/apple/6827677447/
- Foldometer — https://apps.apple.com/app/id6819503409
- foldcounter.com — https://foldcounter.com/